



Rapport NFE204

Analyse des accidents corporels de la circulation survenus en France en 2009

NFE204 – 2016

Lamine BELAHCENE

Mounia BELAHCENE

Laminebelahcene@gmail.com/b.bm@live.com

Table des matières

Introduction	3
I. Présentation et caractéristiques du système Cassandra	4
1. Présentation	4
2. Fonctionnement et modèle de données.....	4
3. Caractéristiques du système Cassandra	4
II. Présentation des données.....	5
1. Source des données	5
2. Description de la base de données.....	5
3. Traitement de la table dans python.....	6
4. Importation des données dans Cassandra	8
5. Requête sur les données.....	11
6. Table d'index	11
III. Cassandra en mode distribué	14
1. Création de l'environnement de travail (répartition sous cassandra)	14
2. Création d'un Keyspace, (réplication sous cassandra).....	15
Conclusion.....	24
Annexe	25

Introduction

Nous nous sommes basé dans notre étude sur le système de gestion des données à grosse volumétrie du système NoSQL Cassandra afin d'importer et analyser les données portant sur les accidents corporels de 2014(soit un accident survenu sur une voie ouverte à la circulation publique, impliquant au moins un véhicule et ayant fait au moins une victime ayant nécessité des soins).

Cette étude sera composée en trois parties, la première où nous présenterons le système Cassandra, son histoire, ses caractéristiques principales et son mode de fonctionnement. La deuxième partie fera l'objet d'une brève introduction de notre base de données de 2014 en France.

Enfin, dans la dernière partie, nous explorerons les possibilités offertes par Cassandra pour faire des requêtes sur notre base de données, indexation pour la recherche d'information afin de construire un moteur de recherche et analyser les bulletins des accidents, déploiement dans une grappe avec réplication de serveur et teste de la reprise sur panne.

I. Présentation et caractéristiques du système Cassandra

1. Présentation

Cassandra a été créée en 2007 par les ingénieurs de Facebook pour palier au problème de stockage des données mais ce n'est qu'en 2009 qu'il a réellement commencé à être commercialisé auprès du grand public.

Par ailleurs, le nom Cassandra nous vient de la mythologie grecque. En effet, Cassandra était la fille du roi Troie qui avait la capacité de voyance d'où le logo du logiciel.

2. Fonctionnement et modèle de données

L'idée est que Cassandra se base sur une denormalisation des données. En effet, les données sont regroupées dans des familles de colonnes que l'on nomme « column families », qui représentent des collections, autrement dit un ensemble de documents. C'est ainsi que nous constituerons notre modèle de données lors de la phase d'écriture qui représente l'étape la plus importante et lourde de l'étude. Ainsi la plus petite unité de notre modèle est la colonne : un triplet contenant un nom, une valeur et un timestamp.

Par ailleurs notre modèle est composé de plusieurs ligne chacune étant composée d'un ensemble de colonnes et est reconnue par une clé. Chaque clé correspond à un document.

3. Caractéristiques du système Cassandra

1. Gestion des pannes : les données d'un nœud sont automatiquement répliquées vers d'autres nœuds issus de différentes machines.
2. Disponibilité/Tuneable Consistency: le niveau de cohérence concernant la lecture et l'écriture peuvent être spécifié en amont, ce qui rend l'écriture des données plus rapide qu'un bon de bases de données.
3. Souplesse/richesse du modèle: le modèle de données repose sur la notion de clé/valeur qui permet de traiter tous les cas figures du Big Data. Les nœuds sont disposés en file, dans un anneau directionnel. L'ajout se fait naturellement à la fin de l'anneau.
4. Décentralisé : contrairement à beaucoup d'autres systemes NoSQL où la notion de cluster maitre/esclave est prédominante. Sur Cassandra, tous les nœuds d'un cluster sont égaux et

ont le même rôle et importance. Par ailleurs la typologie de l'anneau étant connue par tous les nœuds.

5. Scalabilité est linéaire : le débit d'écriture et de lecture augmente de façon linéaire lorsqu'un nouveau serveur est ajouté dans le cluster.

II. Présentation des données

1. Source des données

Les données sont issues de la Plateforme ouverte des données publiques françaises, (<https://www.data.gouv.fr/fr/>) la base qui nous intéresse porte sur les accidents corporels (soit un accident survenu sur une voie ouverte à la circulation publique, impliquant au moins un véhicule et ayant fait au moins une victime ayant nécessité des soins).

Les saisies sont rassemblées dans une fiche intitulée bulletin d'analyse des accidents corporels. L'ensemble de ces fiches constitue le fichier national des accidents corporels de la circulation dit " Fichier BAAC " administré par l'Observatoire national interministériel de la sécurité routière "ONISR".

2. Description de la base de données

Les données 2014 sont désormais annuelles et composées de 4 fichiers (Caractéristiques – Lieux – Véhicules – Usagers) au format csv :

1. La rubrique CARACTERISTIQUES qui décrit les circonstances générales de l'accident
2. La rubrique LIEUX qui décrit le lieu principal de l'accident même si celui-ci s'est déroulé à une
3. intersection
4. La rubrique VEHICULES impliqués
5. La rubrique USAGERS impliqués

Présentation de la table Caractéristiques :

Variables	Libellé	catégorie
Num_Acc	Numéro d'identifiant de l'accident	
jour	Jour de l'accident	
mois	Mois de l'accident	
an	Année de l'accident	
hrmn	Heure et minutes de l'accident	
Heure et minutes de l'accident		
lum	Lumière : conditions d'éclairage dans lesquelles l'accident s'es	1 - Plein jour 2 - Crépuscule ou aube 3 - Nuit sans éclairage public 4 - Nuit avec éclairage public non allumé 5 - Nuit avec éclairage public allumé
dep	Département : Code INSEE du département suivi	
com	Commune : Le numéro de commune est un code donné par l'INSEE. Le code comporte 3 chiffres calés à droite.	
agg	Localisation :	1 - Hors agglomération 2 - En agglomération
Intersection :	Intersection	1 - Hors intersection 2 - Intersection en X 3 - Intersection en T 4 - Intersection en Y 5 - Intersection à plus de 4 branches 6 - Giratoire 7 - Place 8 - Passage à niveau 9 - Autre intersection
atm	Conditions atmosphériques :	1 - Normale 2 - Pluie légère 3 - Pluie forte 4 - Neige - grêle 5 - Brouillard - fumée 6 - Vent fort - tempête 7 - Temps éblouissant 8 - Temps couvert
col	Type de collision :	1 - Deux véhicules - frontale 2 - Deux véhicules - par l'arrière 3 - Deux véhicules - par le côté 4 - Trois véhicules et plus - en chaîne 5 - Trois véhicules et plus - collisions multiples 6 - Autre collision 7 - Sans collision
adr	Adresse postale : variable renseignée pour les accidents survenus en agglomération	
gps	Codage GPS : 1 caractère indicateur de provenance :	M = Métropole A = Antilles (Martinique ou Guadeloupe) G = Guyane R = Réunion Y = Mayotte
lat	Latitude	
long	Longitude	

3. Traitement de la table dans python

On va utiliser python pour faire la jointure entre les tables pour obtenir une base complète avec 132186 lignes et 49 variables après de nombreuses recherches. Nous avons décidé de regrouper nos bases vu que Cassandra ne gère pas les jointures. Par ailleurs, nous n'allons pas convertir le fichier en JSON parce que Cassandra peut directement lire le fichier CSV

Importation de la Table Usagers

```
In [1]: import pandas as pd
users=pd.read_csv("usagers_2014.csv")
users.head(3)
```

```
Out[1]:
```

	Num_Acc	place	catu	grav	sexe	trajet	secu	locp	actp	etatp	an_nais	num_veh
0	201400000001	1.0	1	3	1	5.0	21.0	0.0	0.0	0.0	1971.0	A01
1	201400000001	1.0	1	1	1	5.0	11.0	0.0	0.0	0.0	1992.0	B02
2	201400000002	1.0	1	4	1	5.0	11.0	0.0	0.0	0.0	1983.0	A01

Importation de la Table Caracteristiques

```
In [2]: caracteristiques=pd.read_csv("caracteristiques_2014.csv")
caracteristiques.head(3)
```

```
Out[2]:
```

	Num_Acc	an	mois	jour	hrmn	lum	agg	int	atm	col	com	adr	gps	lat	long	dep	Unnamed: 16
0	201400000001	14	5	7	2015	1	2	1	1.0	3	11	route de don	M	0	0.0	590.0	NaN
1	201400000002	14	5	31	430	1	2	1	1.0	6	11	106 ROUTE DE DON	M	0	0.0	590.0	NaN
2	201400000003	14	8	23	1800	1	2	9	1.0	3	52	75 bis rue jean jaures	M	0	0.0	590.0	NaN

Importation de la Table Lieux

```
In [12]: lieux=pd.read_csv("lieux_2014.csv")
lieux.head(3)
```

```
Out[12]:
```

	Num_Acc	catr	voie	v1	v2	circ	nbv	pr	pr1	vosp	prof	plan	lartpc	larout	surf	infra	situ	env1
0	201400000001	3	41.0	NaN	NaN	2.0	0.0	0.0	0.0	0.0	1.0	1.0	0.0	60.0	1.0	0.0	1.0	0.0
1	201400000002	3	41.0	NaN	NaN	2.0	2.0	0.0	0.0	0.0	1.0	2.0	0.0	62.0	1.0	0.0	4.0	99.0
2	201400000003	3	39.0	NaN	NaN	2.0	0.0	0.0	0.0	0.0	1.0	1.0	0.0	60.0	1.0	0.0	1.0	99.0

Importation de la Table Vehicules

```
In [13]: vehicules=pd.read_csv("vehicules_2014.csv")
vehicules.head()
```

```
Out[13]:
```

	Num_Acc	senc	catv	occutc	obs	obsm	choc	manv	num_veh
0	201400000001	0	33	0	0.0	2.0	1.0	1.0	A01
1	201400000001	0	7	0	0.0	0.0	6.0	15.0	B02
2	201400000002	0	7	0	1.0	0.0	7.0	13.0	A01
3	201400000003	0	2	0	0.0	2.0	1.0	1.0	A01
4	201400000003	0	7	0	0.0	2.0	7.0	15.0	B02

Jointures des tables

```
In [23]: result1 = pd.merge(usagers, vehicules, on=('Num_Acc','num_veh'))
result2 = pd.merge(result1, caracteristiques, on='Num_Acc')
result3 = pd.merge(result2, lieux, on='Num_Acc')
```

exportation en CSV

```
In [22]: result3.to_csv("labase.csv", header=False, index=False)
```

```
In [24]: result3.columns
```

```
Out[24]: Index([u'Num_Acc', u'place', u'catu', u'grav', u'sexe', u'trajet', u'secu',
u'locp', u'actp', u'etatp', u'an_nais', u'num_veh', u'senc', u'catv',
u'occut', u'obs', u'obsm', u'choc', u'manv', u'an', u'mois', u'jour',
u'hrmn', u'lum', u'agg', u'int', u'atm', u'col', u'com', u'adr', u'gps',
u'lat', u'long', u'dep', u'Unnamed: 16', u'catr', u'voie', u'v1', u'v2',
u'circ', u'nbnv', u'pr', u'pr1', u'vosp', u'prof', u'plan', u'lartpc',
u'larrout', u'surf', u'infra', u'situ', u'env1'],
dtype='object')
```

```
In [25]: result3.shape
```

```
Out[25]: (132186, 52)
```

4. Importation des données dans Cassandra

Afin de pouvoir utiliser Cassandra on a installé docker et téléchargé une image docker qu'on appelle Mon_serveur_cassandra avec la commande suivante :

```
docker run --name mon-serveur-cassandra -e "CASSANDRA_TOKEN=1" -d spotify/cassandra:cluster
```

Nous pouvons vérifier le téléchargement de notre image comme ceci avec la commande

```
732f13620e5e  spotify/cassandra:cluster  "cassandra-clusternod"  2 days ago  Up 2 days  22/tcp, 7000-7001/tcp, 7199/tc
p, 8012/tcp, 9042/tcp, 9160/tcp, 61621/tcp  mon-serveur-cassandra
```

docker ps :

On doit récupérer l'adresse IP pour pouvoir lancer ensuite Cassandra Query Language shell(cqlsh)

Commandes :

```
docker inspect mon-serveur-cassandra
```

Réponse :

```
"Aliases": null,
"NetworkID": "621f5d206ed3a37efc194f99e7453759a9049d7b944b39ccde259d9658547456",
"EndpointID": "ed937ce7d068832f0a7073035152200176774e07c483b9e0875bc1bd85e87b19",
"Gateway": "172.17.0.1",
"IPAddress": "172.17.0.2",
```

L'adresse IP est '172.17.0.2'

NFE204 – Bases de données documentaires et distribuées

On envoie le fichier des données dans notre conteneur Cassandra avec la commande suivante :

```
docker cp labase.csv mon-serveur-cassandra:/labase.csv
```

Ensuite on lance le conteneur et on vérifie la réception du fichier:

Commande :

```
docker exec -it mon-serveur-cassandra /bin/bash  
ls
```

```
root@732f13620e5e:/# ls  
bin      caracteristiques_2014.csv  home      lib          media  proc  sbin  tmp      var  
boot     dev                      inside-container-file  lib64      mnt     root  srv    usagers_2014.csv  vehicules.csv  
caracteristiques  etc                      labase.csv  lieux_2014.csv  opt     run   sys    usr          vehicules_2014.csv
```

Réponse :

On constate que le fichier a bien été importé.

Une fois lancé, un terminal dans le conteneur avec la commande précédente, il faut se connecter au serveur Cassandra.

Commande :

```
cqlsh 172.17.0.2
```

On doit créer maintenant notre keyspace ; base de données ; on décide de le faire avec une réplication avec comme facteur 3.

```
CREATE KEYSPACE IF NOT EXISTS Projet  
WITH REPLICATION = { 'class' : 'SimpleStrategy', 'replication_factor': 3 };
```

Pour avoir la liste des keyspace on doit exécuter la commande suivante :

Commande :

```
cqlsh> DESCRIBE keyspaces;
```

Réponse :

```
system projet system_traces demo
```

On peut modifier un keyspace avec la commande :

```
ALTER KEYSPACE
```

Où la commande qui permet de supprimer un keyspace :

```
DROP KEYSPACE
```

On peut accéder au keyspace qu'on a créé avec la commande suivante :

```
DESCRIBE KEYSPACE projet;
```

Réponse :

```
CREATE KEYSPACE projet WITH replication = {  
  'class': 'SimpleStrategy',  
  'replication_factor': '3'  
};
```

Pour l'instant on n'a pas créé de columnfamily mais avec cette commande on peut voir tous nos columnfamily (table) et nos index créés.

La commande TRUNCATE nous permet de tout supprimer.

Pour utiliser notre keyspace on doit exécuter la commande :

```
Use Projet;
```

```
CREATE TABLE base (num_acc bigint, place int, cat int, grav int, sexe int, trajet int, sec int, locp int, actp int, etatp int, an_nais  
int, num_veh varchar, senc int, catv int, occtc int, obs int, obsm int, choc int, manv int, an int, mois int, jour int, hrnm int, lum int, agg  
int, inte int, atm int, col int, com int, adr varchar, gps varchar, lat bigint, long bigint, dep int, catr int, voie int, v1 int, v2 varchar, circ  
int, nbv int, pr float, prl int, vosp int, prof int, plan int, lartpc int, larrou int, surf int, infra int, situ int, envl int, PRIMARY KEY(  
num_acc, num_veh));
```

Création d'une column family

Insertion de données

On doit insérer les données du notre fichier qu'on avait précédemment transféré dans notre machine. Pour ce faire, on utilise la commande copy :

Commande :

```
COPY base (num_acc,place,cat,grav,sexe,trajet,sec,locp,actp,etatp,an_nais,num_veh,senc,catv,occtc,obs,obsm,choc,manv,an,mois,jour,hrmn,lum,agg,inte,atm,col,com,adr,gps,lat,long,dep,catr,voie,v1,v2,circ,nbv,pr,prl,vosp,prof,plan,lartpc,larrout,surf,infra,situ,env1) FROM 'labase.csv';
```

Réponse :

```
132186 rows imported in 3 minutes and 0.837 seconds.
```

Cassandra donne la possibilité de rajouter des données à la main avec la commande :

```
INSERT INTO notre_table (var1, var2, var2)
```

```
VALUES ( val1, 'val2', 'val3');
```

Comme pour les keyspaces on peut modifier et supprimer les tables à l'aide des commandes ci-dessous :

```
ALTER TABLE
```

```
DROP TABLE
```

De plus, on peut faire une mise à jour des données avec UPDATE ou la supprimer avec un DELETE

5. Requête sur les données

Cela ressemble beaucoup à du sql par exemple on peut faire apparaître que les données de l'accident numéro '201400000001' avec la requête suivante :

```
select * from base where num_acc=201400000001;
```

6. Table d'index

Les index intégrés de Cassandra sont les meilleurs sur une table ayant de nombreuses lignes qui contiennent la valeur indexée. Les valeurs les plus uniques qui existent dans une colonne particulière, plus les frais généraux.

Ne pas utiliser un index dans ces situations:

Sur les colonnes de haute cardinalité parce que vous vous interrogez un énorme volume de documents pour un petit nombre de résultats :

Problèmes d'utilisation d'un index de colonne haute cardinalité

Si on crée un index sur une colonne haute cardinalité, qui a beaucoup de valeurs distinctes, une requête entre les champs encourra beaucoup de recherche pour très peu de résultats. Dans le tableau avec un milliard de chansons, à la recherche des chansons de l'écrivain (une valeur qui est généralement unique pour chaque chanson) plutôt que par leur artiste, est susceptible d'être très inefficace. Il serait probablement plus efficace pour maintenir manuellement la table comme une forme d'un index au lieu d'utiliser le Cassandra intégré dans l'indice. Pour les colonnes contenant des données uniques, il est parfois une bonne performance de utiliser un index pour plus de commodité, tant que le volume de requête à la table ayant une colonne indexée est modérée et pas sous charge constante.

A l'inverse, la création d'un index sur une colonne extrêmement faible cardinalité, comme une colonne booléenne, n'a pas de sens. Chaque valeur de l'indice devient une seule ligne dans l'indice, ce qui entraîne une énorme ligne pour toutes les fausses valeurs, par exemple.

Indexation une multitude de colonnes indexées ayant foo = true et foo = false est pas utile.

Dans les tableaux qui utilisent une colonne de compteur

Sur une colonne fréquemment mis à jour ou supprimé.

Pour rechercher une ligne dans une grande partition si étroitement interrogé.

On va créer par exemple un index pour la Catégorie du véhicule :

```
CREATE INDEX catv_id ON base( catv );
```

Ensuite on va afficher les accidents Scooter > 125 cm³ (catv=34)

Commande :

```
SELECT * FROM vehicules WHERE catv =34;
```

Réponse :

NFE204 – Bases de données documentaires et distribuées

```
cqlsh:projet> SELECT * FROM base WHERE catv =34;
```

num_acc	etatp	gps	num_veh	grav	actp	adr	agg	an	an_nais	atm	cat	catr	catv	choc	circ	col	com	dep	env
ctc	place	plan	pr	pr1	prof	sec	senc	sexe	situ	surf	trajet	v1	v2	voie	vosp	nbv	obs	obsn	oc
201400027681	0	0	2	1	3	2209	0	1	15	0	2	14	1996	1	2	4	34	7	0
0	0	2	1	3	2209	0	1	15	0	2	14	1996	1	2	4	34	7	0	0
201400052459	0	0	1	1	4	1215	0	3	23	125	0	1	1991	1	1	4	34	1	2
0	0	1	1	4	1215	0	3	23	125	0	1	1991	1	1	4	34	1	2	106
201400055489	0	0	1	1	4	1950	0	1	21	0	1	14	1989	1	2	4	34	2	1
0	0	1	1	4	1950	0	1	21	0	1	14	1989	1	2	4	34	2	1	116
201400055489	0	0	1	1	4	1950	0	1	21	0	1	14	1965	1	1	4	34	6	1
0	0	1	1	4	1950	0	1	21	0	1	14	1965	1	1	4	34	6	1	116
201400013671	0	0	M	3	1800	0	5	2	31	95	2	14	1985	1	2	3	34	6	2
0	0	M	3	1800	0	5	2	31	95	2	14	1985	1	2	3	34	6	2	130
201400055696	0	0	1	1	4	1110	0	1	21	0	2	14	1986	1	1	4	34	1	1
0	0	1	1	4	1110	0	1	21	0	2	14	1986	1	1	4	34	1	1	116
201400043984	0	0	M	4	1200	0	0	1	27	0	2	14	1985	1	3	3	34	4	1
0	0	M	4	1200	0	0	1	27	0	2	14	1985	1	3	3	34	4	1	18
201400045449	0	0	1	1	4	1815	0	1	22	0	1	14	1974	1	1	3	34	1	2
0	0	1	1	4	1815	0	1	22	0	1	14	1974	1	1	3	34	1	2	80
201400041888	0	0	1	1	4	1245	0	1	10	0	2	14	1976	8	1	2	34	2	2
0	0	1	1	4	1245	0	1	10	0	2	14	1976	8	1	2	34	2	2	10
0	0	1	1	4	1245	0	1	10	0	2	14	1976	8	1	2	34	2	2	930
0	0	1	1	4	1245	0	1	10	0	2	14	1976	8	1	2	34	2	2	2

Et bien sûr on peut faire des recherches multicritères pour affiner les recherches :

Commandes :

```
select * from base where num_acc=201400000001 and catv=33
```

Résultat:

```
cqlsh:projet> select * from base where num_acc=201400000001 and catv=33 ;
```

num_acc	etatp	gps	num_veh	grav	actp	adr	agg	an	an_nais	atm	cat	catr	catv	choc	circ	col	com	dep	env
ps	grav	hrmn	infra	inte	jour	larout	lartpc	lat	locp	long	lum	manv	mois	nbv	obs	obsn	occtc	place	plan
201400000001	0	0	3	2015	0	1	7	60	0	0	0	0	1	1	5	0	0	2	0
0	0	3	2015	0	1	7	60	0	0	0	0	1	1	5	0	0	2	0	1
0	0	1	21	0	1	1	1	5	null	null	41	0	0	0	0	0	0	0	0

(1 rows)

Index multiples :

On peut utiliser plusieurs index pour nos études, par exemple on va créer des index pour la variable de gravité de l'accident(Grav) et la variable Catégorie de route(catr) pour afficher que les personnes morte(grav=2) sur l'autoroute(catr=1)

```
CREATE INDEX grav_id on base(grav);
```

```
CREATE INDEX catr_id on base(catr);
```

```
Select * from base Where grav=2 and catr=1
```

III. Cassandra en mode distribué

Voyons maintenant le mode distribué de Cassandra sur nos données pour cela on va créer 1 cluster avec 5 nœuds. Chaque nœud est indépendant avec ses propres données, son IP...

1. Création de l'environnement de travail (répartition sous cassandra)

Dans ce qui suit nous allons créer 5 nœuds avec un token CASSANDRA_TOKEN qui permet comme expliqué dans la partie théorique de respecter l'architecture et les placer sur le Hash Ring. Chacun des 5 nœuds est responsable d'un intervalle de valeurs de hachage sur l'anneau à l'aide des partitioners. En effet ces derniers repartissent de manière uniforme notre base donnée sur les 5 nœuds.

Il faut démarrer des instances de Cassandra afin d'obtenir un cluster sur le port de Cassandra-1 dans notre cas c'est '172.17.0.2'.

Le démarrage d'un nouveau nœud Cassandra passe encore par l'utilisation de la même image avec la commande suivante :

```
docker run -d -e "CASSANDRA_TOKEN=10" -e "CASSANDRA_SEEDS=172.17.0.2" --name cassandra-2 spotify/cassandra:cluster
docker run -d -e "CASSANDRA_TOKEN=100" -e "CASSANDRA_SEEDS=172.17.0.2" --name cassandra-3 spotify/cassandra:cluster
docker run -d -e "CASSANDRA_TOKEN=1000" -e "CASSANDRA_SEEDS=172.17.0.2" --name cassandra-4 spotify/cassandra:cluster
docker run -d -e "CASSANDRA_TOKEN=10000" -e "CASSANDRA_SEEDS=172.17.0.2" --name cassandra-5 spotify/cassandra:cluster
```

On voit voir maintenant l'état du cluster avant l'insertion des données avec la commande `nodetool ring` :

```
lamine@lamine-Lenovo-G580:~$ sudo docker exec -it cassandra-1 /bin/bash
root@27281a2c38c6:/# nodetool ring

Datacenter: datacenter1
=====
Address      Rack      Status State  Load            Owns               Token
-----
172.17.0.2   rack1     Up      Normal  36.16 KB        100.00%           1
172.17.0.3   rack1     Up      Normal  45.38 KB        100.00%           10
172.17.0.4   rack1     Up      Normal  77.94 KB        0.00%             100
172.17.0.5   rack1     Up      Normal  45.42 KB        0.00%             1000
172.17.0.6   rack1     Up      Normal  49.27 KB        0.00%             10000
```

On constate qu'on a 5 nœuds avec des volumes faibles 6,16KB, 45,38KB, 77,94KB, 45,42KB, 49,27KB vu qu'il y a pas encore de données.

2. Création d'un Keyspace, (réplication sous cassandra)

Maintenant on va créer un keyspace qu'on va appeler distrib avec un facteur de réplication de 3.

C'est à dire qu'on va répliquer nos données sur 3 nœuds.

Ce facteur est de 1 par défaut mais dans notre cas on a choisi de répliquer nos données dans 2 nœuds après les avoir écrit dans un nœud :

```
docker exec -it cassandra-1 /bin/bash
cqlsh 172.17.0.2
#crée une keyspace avec replication
CREATE keyspace distrib
  with replication = {'class':'SimpleStrategy', 'replication_factor':3};
```

Par ailleurs, nous utilisons ici une stratégie de réplication simple qui consiste à placer nos données sur un nœud choisi par le partitioner. Quant aux répliquions sur les nœuds suivants de l'anneau sans tenir compte de la topologie. Cette dernière ; réplication par topologie ; est utilisée quand on est sur plusieurs clusters où on préfère interroger les nœuds locaux. Le système sera certes reparti sur deux datacenter ou plus néanmoins l'architecture est la même que pour la stratégie simple ; anneau directionnel sur lequel utilisé pour répliquer les données selon sa direction.

On crée ensuite notre Table qu'on appellera 'Base' avec comme clé de table les deux variables suivantes :

Numéro d'accident :num_acc.

Numéro de véhicule :num_veh.

On envoie la base de données sur notre cluster avec la commande CP.

Ensuite avec la commande copy on va insérer les données dans la base créée précédemment

On peut voir le code de ces étapes dans la capture d'écran de Docker :

```
docker cp labase.csv cassandra-1:/labase.csv

docker exec -it cassandra-1 /bin/bash
cqlsh 172.17.0.2
#crée une keyspace avec replication
CREATE keyspace distrib
  with replication = {'class':'SimpleStrategy', 'replication_factor':3};

USE distrib;

CREATE TABLE base (num_acc bigint,place int,cat int,grav int,sexe int,trajet int,sec int,locp int,actp int ,etstp int ,an_nais
int ,num_veh varchar,senc int,catv int,occtc int,obs int,obsm int,choc int,manv int,an int,mois int,jour int,hrmn int,lum int,agg
int,inte int,atm int,col int,com int,adr varchar,gps varchar,lat bigint,long bigint,dep int,catr int,voie int,v1 int,v2 varchar,circ
int,nbv int,pr float,prl int,vosp int,prof int,plan int,lartpc int,larrouit int,surf int,infra int,situ int,env1 int,PRIMARY KEY(
num_acc,num_veh));
#importation des données

COPY base (num_acc,place,cat,grav,sexe,trajet,sec,locp,actp,etstp,an_nais,num_veh,senc,catv,occtc,obs,obsm,choc,
manv,an,mois,jour,hrmn,lum,agg,inte,atm,col,com,adr,gps,lat,long,dep,catr,voie,v1,v2,circ,nbv,pr,
prl,vosp,prof,plan,lartpc,larrouit,surf,infra,situ,env1) FROM 'labase.csv';
```

On constate qu'on a importé 132186 lignes de la base et qu'on n'a pas de rejet ou d'erreur :

```
<e int,trajet int,sec int,locp int,actp int ,etstp int ,an_nais
<t,choc int,manv int,an int,mois int,jour int,hrmn int,lum int,agg
<,infra int,situ int,env1 int,PRIMARY KEY(num_acc,num_veh));
<,trajet,sec,locp,actp,etstp,an_nais,num_veh,senc,catv,occtc,obs,obsm,choc
<m,col,com,adr,gps,lat,long,dep,catr,voie,v1,v2,circ,nbv,pr,
<f,plan,lartpc,larrouit,surf,infra,situ,env1) FROM 'labase.csv';
Can't open 'labase.csv' for reading: [Errno 2] No such file or directory:
e.csv'
0 rows imported in 0.000 seconds.
<pc,larrouit,surf,infra,situ,env1) FROM 'labase.csv';

132186 rows imported in 3 minutes and 52.917 seconds.
```

État du cluster après l'insertion des données :

cluster-1:

```
lamine@lamine-Lenovo-G580:~$ sudo docker exec -it cassandra-1 /bin/bash
[sudo] Mot de passe de lamine :
root@27281a2c38c6:/# nodetool cfstats -h 172.17.0.2 distrib
Keyspace: distrib
  Read Count: 0
  Read Latency: NaN ms.
  Write Count: 132186
  Write Latency: 0.2083994598520267 ms.
  Pending Tasks: 0
    Table: base
      SSTable count: 3
      Space used (live), bytes: 52804109
      Space used (total), bytes: 52820493
      SSTable Compression Ratio: 0.33769762440400614
      Number of keys (estimate): 56192
      Memtable cell count: 407750
      Memtable data size, bytes: 61973719
      Memtable switch count: 12
      Local read count: 0
      Local read latency: 0.000 ms
      Local write count: 132186
      Local write latency: 0.208 ms
      Pending tasks: 0
      Bloom filter false positives: 0
      Bloom filter false ratio: 0.00000
      Bloom filter space used, bytes: 71272
      Compacted partition minimum bytes: 1598
      Compacted partition maximum bytes: 20501
      Compacted partition mean bytes: 2920
      Average live cells per slice (last five minutes): 0.0
      Average tombstones per slice (last five minutes): 0.0
```

cluster 2 :

```
lamine@lamine-Lenovo-G580:~$ sudo docker exec -it cassandra-2 /bin/bash
[sudo] Mot de passe de lamine :
root@ade8d1de4f38:/# nodetool cfstats -h 172.17.0.3 distrib
Keyspace: distrib
  Read Count: 0
  Read Latency: NaN ms.
  Write Count: 132186
  Write Latency: 0.18063312302361823 ms.
  Pending Tasks: 0
    Table: base
      SSTable count: 3
      Space used (live), bytes: 52508218
      Space used (total), bytes: 52524602
      SSTable Compression Ratio: 0.3379121368467095
      Number of keys (estimate): 55936
      Memtable cell count: 441600
      Memtable data size, bytes: 66941950
      Memtable switch count: 12
      Local read count: 0
      Local read latency: 0.000 ms
      Local write count: 132186
      Local write latency: 0.181 ms
      Pending tasks: 0
      Bloom filter false positives: 0
      Bloom filter false ratio: 0.00000
      Bloom filter space used, bytes: 70608
      Compacted partition minimum bytes: 1598
      Compacted partition maximum bytes: 20501
      Compacted partition mean bytes: 2920
      Average live cells per slice (last five minutes): 0.0
      Average tombstones per slice (last five minutes): 0.0
```

cluster 3:

```
lamine@lamine-Lenovo-G580:~$ sudo docker exec -it cassandra-3 /bin/bash
[sudo] Mot de passe de lamine :
root@8f6e5877eae5:/# nodetool cfstats -h 172.17.0.4 distrib
Keyspace: distrib
  Read Count: 0
  Read Latency: NaN ms.
  Write Count: 132186
  Write Latency: 0.18621351731650856 ms.
  Pending Tasks: 0
    Table: base
      SSTable count: 3
      Space used (live), bytes: 52646272
      Space used (total), bytes: 52662676
      SSTable Compression Ratio: 0.33783298748266843
      Number of keys (estimate): 55936
      Memtable cell count: 425850
      Memtable data size, bytes: 63760067
      Memtable switch count: 12
      Local read count: 0
      Local read latency: 0.000 ms
      Local write count: 132186
      Local write latency: 0.186 ms
      Pending tasks: 0
      Bloom filter false positives: 0
      Bloom filter false ratio: 0.00000
      Bloom filter space used, bytes: 70880
      Compacted partition minimum bytes: 1598
      Compacted partition maximum bytes: 20501
      Compacted partition mean bytes: 2921
      Average live cells per slice (last five minutes): 0.0
      Average tombstones per slice (last five minutes): 0.0
```

cluster 4 :

```
lamine@lamine-Lenovo-G580:~$ sudo docker exec -it cassandra-4 /bin/bash
[sudo] Mot de passe de lamine :
Désolé, essayez de nouveau.
[sudo] Mot de passe de lamine :
root@ab7913d17fd1:/# nodetool cfstats -h 172.17.0.5 distrib
Keyspace: distrib
  Read Count: 0
  Read Latency: NaN ms.
  Write Count: 0
  Write Latency: NaN ms.
  Pending Tasks: 0
    Table: base
      SSTable count: 0
      Space used (live), bytes: 0
      Space used (total), bytes: 0
      SSTable Compression Ratio: 0.0
      Number of keys (estimate): 0
      Memtable cell count: 0
      Memtable data size, bytes: 0
      Memtable switch count: 0
      Local read count: 0
      Local read latency: 0.000 ms
      Local write count: 0
      Local write latency: 0.000 ms
      Pending tasks: 0
      Bloom filter false positives: 0
      Bloom filter false ratio: 0.00000
      Bloom filter space used, bytes: 0
      Compacted partition minimum bytes: 0
      Compacted partition maximum bytes: 0
      Compacted partition mean bytes: 0
      Average live cells per slice (last five minutes): 0.0
      Average tombstones per slice (last five minutes): 0.0
```

cluster 5 :

```
lamine@lamine-Lenovo-G580:~$ sudo docker exec -it cassandra-5 /bin/bash
[sudo] Mot de passe de lamine :
root@fbf472567bae:/# nodetool cfstats -h 172.17.0.6 distrib
Keyspace: distrib
  Read Count: 0
  Read Latency: NaN ms.
  Write Count: 0
  Write Latency: NaN ms.
  Pending Tasks: 0
    Table: base
      SSTable count: 0
      Space used (live), bytes: 0
      Space used (total), bytes: 0
      SSTable Compression Ratio: 0.0
      Number of keys (estimate): 0
      Memtable cell count: 0
      Memtable data size, bytes: 0
      Memtable switch count: 0
      Local read count: 0
      Local read latency: 0.000 ms
      Local write count: 0
      Local write latency: 0.000 ms
      Pending tasks: 0
      Bloom filter false positives: 0
      Bloom filter false ratio: 0.00000
      Bloom filter space used, bytes: 0
      Compacted partition minimum bytes: 0
      Compacted partition maximum bytes: 0
      Compacted partition mean bytes: 0
      Average live cells per slice (last five minutes): 0.0
      Average tombstones per slice (last five minutes): 0.0
```

On voit bien que la données est répliqué sur le cluster-1,cluster-2 et cluster-3 comme on l'a décrit sur le Keyspace avec un write count de 12186 et que le cluster 4 et cluster 5 ne contienne pas de données(write count =0).

Nodetool ring après insertion des données :

```
lamine@lamine-Lenovo-G580:~$ sudo docker exec -it cassandra-1 /bin/bash
[sudo] Mot de passe de lamine :
root@27281a2c38c6:/# nodetool ring
Note: Ownership information does not include topology; for complete information,
specify a keyspace

Datacenter: datacenter1
=====
Address      Rack      Status State  Load        Owns          Token
-----
172.17.0.2   rack1     Up      Normal  50.42 MB     100.00%       1
172.17.0.3   rack1     Up      Normal  50.16 MB     0.00%         10
172.17.0.4   rack1     Up      Normal  50.28 MB     0.00%        100
172.17.0.5   rack1     Up      Normal  68.2 KB      0.00%       1000
172.17.0.6   rack1     Up      Normal  72.06 KB     0.00%      10000
```



```
lamine@lamine-Lenovo-G580:~$ sudo docker exec -it cassandra-1 /bin/bash
root@27281a2c38c6:/# nodetool ring
Note: Ownership information does not include topology; for complete information,
specify a keyspace

Datacenter: datacenter1
=====
Address      Rack      Status State  Load             Owns               Token
-----
172.17.0.2   rack1     Up      Normal  50.42 MB         100.00%            1
172.17.0.3   rack1     Down    Normal  50.16 MB         0.00%              10
172.17.0.4   rack1     Down    Normal  50.28 MB         0.00%              100
172.17.0.5   rack1     Up      Normal  68.2 KB          0.00%              1000
172.17.0.6   rack1     Up      Normal  72.06 KB         0.00%              10000
```

Il existe des stratégies de cohérence en écriture et en lecture qui vont soit optimiser soit la disponibilité du système, soit maximiser la cohérence des données.

Nous allons réaliser des requêtes avec le paramétrés ALL, ONE et QUORUM.

Pour le ALL :

Comme vu lors de sa définition le mode all reçoit la réponse de tous les répliquats, et comme 2 répliquats ne répondent pas, la requête est en échec.

```
root@27281a2c38c6:/# cqlsh 172.17.0.2
Connected to Test Cluster at 172.17.0.2:9160.
[cqlsh 4.1.1 | Cassandra 2.0.10 | CQL spec 3.1.1 | Thrift protocol 19.39.0]
Use HELP for help.
cqlsh> use distrib;
cqlsh:distrib> consistency all;
Consistency level set to ALL.
cqlsh:distrib> select * from base limit 2 ;
Unable to complete request: one or more nodes were unavailable.
```

Pour le ONE :

Le mode ONE, comme on le voit le coordinateur reçoit la réponse la plus proche et la renvoie au client les données du noeud cluster-1.

NFE204 – Bases de données documentaires et distribuées

```
cqlsh:distrib> consistency one;
consistency level set to ONE.
cqlsh:distrib> select * from base limit 2 ;
```

num_acc	num_veh	actp	adr	agg	an	an_nais	atm	cat	catr	catv	choc	circ	col	com	dep	env1	et					
atp	gps	grav	hrmn	infra	inte	jour	larrou	lartpc	lat	locp	long	lum	manv	mois	nbv	obs	obsm					
plan	pr	pr1	prof	sec	senc	sexe	situ	surf	trajet	v1	v2	voie	vosp									
201400059722		A01		0			RN3 (1)	1	14		1994	1	1	2	7	9	1	7	16	974	99	
0	R	4	700		1	30		0		0	0	0	2	14	6	2	0	0	0	0	1	
1	56		0	2	11	0	1	3	1		0	null	null	3	0							
201400027683		A01		0	139	ROUTE DE VERTOU		2	14		1997	1	1	4	2	1	2	6	109	440	0	
0	null	3	1240		0	1	18	0		0	null	0	null	1	0	5	0	0	2	0	0	1
1	null	null		1	21	0	1	1	1		0	null	null	null	0							

(2 rows)

On constate que la requête passe et on a pas d'erreur et elle nous renvoie bien les données du cluster-1

Pour le Quorum :

Le Quorum est une alternative entre One et ALL il juge du nombre de nœuds activés pour voir s'il voit s'il renvoie les données ou pas.

Avec 2 cluster en pause sur 3 :

```
cqlsh:distrib> consistency quorum;
consistency level set to QUORUM.
cqlsh:distrib> select * from base limit 2 ;
Unable to complete request: one or more nodes were unavailable.
```

Le paramètre quorum nous renvoie aussi comme attendu un message d'erreur pour nous dire que les données sont distribuées sur plusieurs nœuds parce que on a 2 nœuds sur 3 en pause Cassandra juge qu'il ne peut pas envoyer les données

Avec 1 cluster en pause sur 3 :

On va essayer d'activer un autre cluster pour voir si cassandra renvoie des données avec la commande :

```
sudo docker unpause cassandra-2
```

On voit avec le nodetool ring qu'il reste un nœud en pause sur trois :

```
lamine@lamine-Lenovo-G580:~$ sudo docker exec -it cassandra-1 /bin/bash
root@27281a2c38c6:/# nodetool ring
Note: Ownership information does not include topology; for complete information,
specify a keyspace

Datacenter: datacenter1
=====
Address      Rack      Status State      Load           Owns             Token
-----
172.17.0.2   rack1     Up      Normal    50.45 MB       100.00%          1
172.17.0.3   rack1     Up      Normal    50.16 MB       0.00%            10
172.17.0.4   rack1     Down    Normal    50.28 MB       0.00%            100
172.17.0.5   rack1     Up      Normal    69.09 KB       0.00%            1000
172.17.0.6   rack1     Up      Normal    73.25 KB       0.00%            10000
```

On va essayer maintenant d'effectuer une requête avec le paramètre QUORUM :

```
lamine@lamine-Lenovo-G580:~$ sudo docker exec -it cassandra-1 /bin/bash
[sudo] Mot de passe de lamine :
root@27281a2c38c6:/# cqlsh 172.17.0.2
Connected to Test Cluster at 172.17.0.2:9160.
[cqlsh 4.1.1 | Cassandra 2.0.10 | CQL spec 3.1.1 | Thrift protocol 19.39.0]
Use HELP for help.
cqlsh> use distrib;
cqlsh:distrib> consistency quorum;
Consistency level set to QUORUM.
cqlsh:distrib> select * from base limit 2 ;

num_acc | num_veh | actp | adr | agg | an | an_nais | atm | cat | catr | catv | choc | circ | col | com | dep | env1 | et
atp | gps | grav | hrnn | infra | inte | jour | larrout | lartpc | lat | locp | long | lum | manv | mois | nbv | obs | obsm | occtc | place |
plan | pr | pr1 | prof | sec | senc | sexe | situ | surf | trajet | v1 | v2 | voie | vosp
-----+-----
201400059722 | A01 | 0 | RN3 (1) | 1 | 14 | 1994 | 1 | 1 | 2 | 7 | 9 | 1 | 7 | 16 | 974 | 99 |
0 | R | 4 | 700 | 0 | 1 | 30 | 0 | 0 | 0 | 2 | 14 | 6 | 2 | 0 | 0 | 0 | 1 |
1 | 56 | 0 | 2 | 11 | 0 | 1 | 3 | 1 | 0 | null | null | 3 | 0 |
201400027683 | A01 | 0 | 139 ROUTE DE VERTOU | 2 | 14 | 1997 | 1 | 1 | 4 | 2 | 1 | 2 | 6 | 109 | 440 | 0 |
0 | null | 3 | 1240 | 0 | 1 | 18 | 0 | 0 | null | 0 | null | 1 | 0 | 5 | 0 | 0 | 2 | 0 | 1 |
1 | null | null | 1 | 21 | 0 | 1 | 1 | 1 | 0 | null | null | null | 0 |

(2 rows)
```

Dans ce cas avec 2 nœuds sur 3 disponible, la règle quorum est validée et cassandra nous renvoie un résultat.

Conclusion

Suite à l'étude effectuée avec Cassandra, nous avons constaté au-delà de la simplicité de son utilisation, Cassandra convient tout à fait à la gestion d'une grosse volumétrie des données. Et ce de par son architecture qui requiert une distribution des données sur plusieurs nœuds faciles et compatibles avec un environnement distribué de type et les données Big Data.

Néanmoins malgré cet avantage non négligeable, il reste que le modèle de données doit être soigneusement choisi et exige une connaissance fonctionnelle des données. En effet, il est nécessaire de savoir répartir les données sur les bons clusters en amont lors de la modélisation. En conséquence la probabilité d'obtenir des résultats aberrants risque d'être très conséquente. De plus il y a une limitation concernant les tailles des colonnes où les données pour la valeur d'une clé doit être intégrée entièrement dans le disque d'une machine.

Comme constaté lors de notre étude, Cassandra est reconnue aujourd'hui comme étant une des meilleures bases de données NoSQL sous réserve d'application d'une bonne répartition des données sur les différents nœuds/clusters/serveurs.

Annexe

```
docker run --name mon-serveur-cassandra -e "CASSANDRA_TOKEN=1" -d  
spotify/cassandra:cluster
```

#envoyer les fichier dans docker

```
docker cp labase.csv mon-serveur-cassandra:/labase.csv
```

#connexion

```
docker exec -it mon-serveur-cassandra /bin/bash
```

#connexion au serveur Casandra

```
cqlsh 172.17.0.2
```

#création de la base de données

```
CREATE KEYSPACE IF NOT EXISTS Projet
```

```
WITH REPLICATION = { 'class' : 'SimpleStrategy', 'replication_factor': 3 };
```

#Une fois le keyspace créé, essayez les commandes suivantes.

```
SELECT * FROM system.schema_keyspaces;
```

```
cqlsh > DESCRIBE keyspaces;
```

```
cqlsh > DESCRIBE KEYSPACE projet;
```

#Utilisation de la base

```
Use Projet;
```

```
201400048334
```

#crée les tables

```
CREATE TABLE base (num_Acc bigint,place int,cat int,grav int,sexe int,trajet int,sec  
int,locp int,actp int ,etatp int ,an_nais
```

```
int ,num_veh varchar,senc int,catv int,occtc int,obs int,obsm int,choc int,manv int,an int,mois  
int,jour int,hrmn int,lum int,agg
```

NFE204 – Bases de données documentaires et distribuées

```
int,inte int,atm int,col int,com int,adr varchar,gps varchar,lat bigint,long bigint,dep int,catr
int,voie int,v1 int,v2 varchar,circ int,nbv int,pr float,pr1 int,vosp int,prof int,plan int,lartpc
int,larrout int,surf int,infra int,situ int,env1 int,PRIMARY KEY(num_acc,num_veh));
```

#importation des données

COPY base

```
(num_acc,place,cat,grav,sexe,trajet,sec,locp,actp,etatp,an_nais,num_veh,senc,catv,occtc,obs,o
bsm,choc,
```

```
manv,an,mois,jour,hrmn,lum,agg,inte,atm,col,com,adr,gps,lat,long,dep,catr,voie,v1,v2,circ,nb
v,pr,
```

```
pr1,vosp,prof,plan,lartpc,larrout,surf,infra,situ,env1) FROM 'labase.csv';
```

#interrogation

```
CREATE INDEX catv ON base( catv );
```

```
SELECT * FROM base WHERE catv =34;
```

```
select * from base where num_acc=201400000001 and catv=33
```

#index multiples

```
CREATE INDEX grav_id on base(grav);
```

```
CREATE INDEX catr_id on base(catr);
```

```
Select * from base Where grav=2 and catr=1
```

#mode distribué

#6 noeuds

```
sudo docker run -d -e "CASSANDRA_TOKEN=1" --name cassandra-1
spotify/cassandra:cluster
```

```
sudo docker inspect -f '{{.NetworkSettings.IPAddress}}' cassandra-1
```

```
#'172.17.0.2'
```

```
docker run -d -e "CASSANDRA_TOKEN=10" -e "CASSANDRA_SEEDS=172.17.0.2" --
name cassandra-2 spotify/cassandra:cluster
```

NFE204 – Bases de données documentaires et distribuées

```
sudo docker run -d -e "CASSANDRA_TOKEN=100" -e
"CASSANDRA_SEEDS=172.17.0.2" --name cassandra-3 spotify/cassandra:cluster

sudo docker run -d -e "CASSANDRA_TOKEN=1000" -e
"CASSANDRA_SEEDS=172.17.0.2" --name cassandra-4 spotify/cassandra:cluster

sudo docker run -d -e "CASSANDRA_TOKEN=10000" -e
"CASSANDRA_SEEDS=172.17.0.2" --name cassandra-5 spotify/cassandra:cluster

sudo docker exec -it cassandra-1 /bin/bash

nodetool ring

#Création d'un Keyspace, avec un facteur de réplication de 3

cqlsh 172.17.0.2

CREATE keyspace distrib with replication = {'class':'SimpleStrategy', 'replication_factor':3};

#Ajout de données

USE distrib;

CREATE TABLE base (num_Acc bigint,place int,cat int,grav int,sexe int,trajet int,sec
int,locp int,actp int ,etatp int ,an_nais

int ,num_veh varchar,senc int,catv int,occtc int,obs int,obsm int,choc int,manv int,an int,mois
int,jour int,hrmn int,lum int,agg

int,inte int,atm int,col int,com int,adr varchar,gps varchar,lat bigint,long bigint,dep int,catr
int,voie int,v1 int,v2 varchar,circ int,nbv int,pr float,pr1 int,vosp int,prof int,plan int,lartpc
int,larrou int,surf int,infra int,situ int,env1 int,PRIMARY KEY(num_acc,num_veh));

#insertion des données:

COPY base
(num_acc,place,cat,grav,sexe,trajet,sec,locp,actp,etatp,an_nais,num_veh,senc,catv,occtc,obs,o
bsm,choc,

manv,an,mois,jour,hrmn,lum,agg,inte,atm,col,com,adr,gps,lat,long,dep,catr,voie,v1,v2,circ,nb
v,pr,

pr1,vosp,prof,plan,lartpc,larrou,surf,infra,situ,env1) FROM 'labase.csv';
```

NFE204 – Bases de données documentaires et distribuées

Etat des clusters après insertion d'un document

```
sudo docker exec -it cassandra-1 /bin/bash
```

```
nodetool cfstats -h 172.17.0.2 distrib
```

```
sudo docker exec -it cassandra-2 /bin/bash
```

```
nodetool cfstats -h 172.17.0.3 distrib
```

```
sudo docker exec -it cassandra-3 /bin/bash
```

```
nodetool cfstats -h 172.17.0.4 distrib
```

```
sudo docker exec -it cassandra-4 /bin/bash
```

```
nodetool cfstats -h 172.17.0.5 distrib
```

```
sudo docker exec -it cassandra-5 /bin/bash
```

```
nodetool cfstats -h 172.17.0.6 distrib
```

#Hash Ring

```
sudo docker exec -it cassandra-5 /bin/bash
```

```
cqlsh 172.17.0.6
```

```
use distrib;
```

```
select * from base limit 2 ;
```

#Cohérence des données en lecture

```
sudo docker pause cassandra-2
```

```
sudo docker pause cassandra-3
```

```
sudo docker exec -it cassandra-1 /bin/bash
```

```
nodetool ring
```

#all

```
sudo docker exec -it cassandra-1 /bin/bash
```

```
cqlsh 172.17.0.2
```

NFE204 – Bases de données documentaires et distribuées

use distrib;

consistency all;

select * from base limit 2 ;

#one

consistency one;

select * from base limit 2 ;

#quorum

#2 cluster en pause

consistency quorum;

select * from base limit 2 ;

#1 cluster en pause

sudo docker unpause cassandra-2

sudo docker exec -it cassandra-1 /bin/bash

cqlsh 172.17.0.2

use distrib;

consistency quorum;

select * from base limit 2 ;